Report: Agentic AI for Business and FinTech (FTEC5660) – Homework 01

Student Name: ZHANG, Kaiyu
Student ID: 1155238565
Date: January 28, 2026

1. Introduction

The objective of this assignment is to develop an Agentic AI solution capable of analyzing supermarket receipt images and answering specific user queries regarding detailed expenses. Leveraging the multimodal capabilities of Large Language Models (LLMs), the system is designed to extract financial data from visual inputs and perform calculations based on natural language instructions. The core functional requirements involve handling three distinct scenarios: calculating the total final spend after discounts, computing the original price before any savings, and strictly rejecting queries unrelated to the provided receipts. This report documents the implementation strategy, model configuration, and evaluation results of the proposed solution.

2. Methodology

The solution is implemented using Google's Gemini 3 Flash Preview ("gemini-3-flash-preview"), selected for its efficient multimodal processing and reasoning capabilities. The interaction is managed through the `google.genai` Python SDK, allowing for seamless integration of image processing and text generation. The system dynamically loads receipt images from a local directory, converts them into the required `types.Part` format, and processes them alongside the user's text query to generate a response.

The core logic of the agent is governed by a strictly defined system prompt that functions as both a "Gatekeeper" and a "Calculator." This prompt enforces specific behavioral rules aligned with the assignment requirements. Specifically, it instructs the model to sum the *Final Amount Paid* when the user requests the "total spend" or "final bill," ensuring all discounts are accounted for. Conversely, when the user asks for the "price without discount," the prompt directs the model to aggregate the *Subtotal* (price before savings or coupons). Crucially, a guardrail is implemented to identify out-of-domain requests; if a query is unrelated to the receipts (e.g., general knowledge questions), the model is programmed to output a standardized rejection message, ensuring the agent remains focused on its designated task.

To ensure reliability in financial calculations, the generation configuration uses a temperature setting of 0.0. This parameter setting minimizes the model's creativity, enforcing deterministic outputs and reducing the risk of hallucinations when extracting numerical data from the images.

Error handling is also integrated to verify the presence of valid image inputs before executing API calls, preventing unnecessary failures during execution.

3. Experiments and Results

The solution was evaluated using the provided dataset of receipt images. Below are the results from the test execution, demonstrating the model's ability to handle all three required scenarios as documented in the codebase.

3.1 Test Case 1: Total Spend (Query 1)

- **Input Query:** "'How much money did I spend in total for these bills?'"
- **Requirement:** Calculate the final sum paid after discounts.
- **Model Output:**

> Receipt 1: $394.70 + Receipt 2: $316.10 + Receipt 3: $140.80 + Receipt 4: $514.00 + Receipt 5: $102.30 + Receipt 6: $190.80 + Receipt 7: $315.60
> **Total Spend: $1,974.30**

- **Analysis:** The model successfully identified the "Net Total" or final payment line from 7 different receipts and computed the correct sum.

3.2 Test Case 2: Original Price (Query 2)

- **Input Query:** "'How much would I have had to pay without the discount?'"
- **Requirement:** Calculate the subtotal before any savings.
- **Model Output:**

> Calculation logic: Receipt 1: $24.90 + … [detailed breakdown of non-discounted items]

- **Analysis:** The model differentiated between the "Final Amount" and the "Subtotal," correctly aggregating the pre-discount prices as instructed by the system prompt.

3.3 Test Case 3: Out-of-Domain Query

- **Input Query:** "'What is the capital of France?'"
- **Requirement:** Reject the query.
- **Model Output:**

> "I can only analyze requests related to these supermarket receipts."

- **Analysis:** The system successfully triggered the irrelevant query guardrail and refused to answer, adhering to the safety guidelines defined in the system instruction.

4. Conclusion

In conclusion, the implemented solution successfully meets all functional requirements of the assignment. By combining the multimodal capabilities of “gemini-3-flash-preview” with precise prompt engineering and deterministic configuration settings, the agent reliably distinguishes between different financial metrics such as net versus gross costs. Furthermore, the system maintains strict domain boundaries, effectively filtering out irrelevant user queries while providing accurate and verifiable financial summaries.